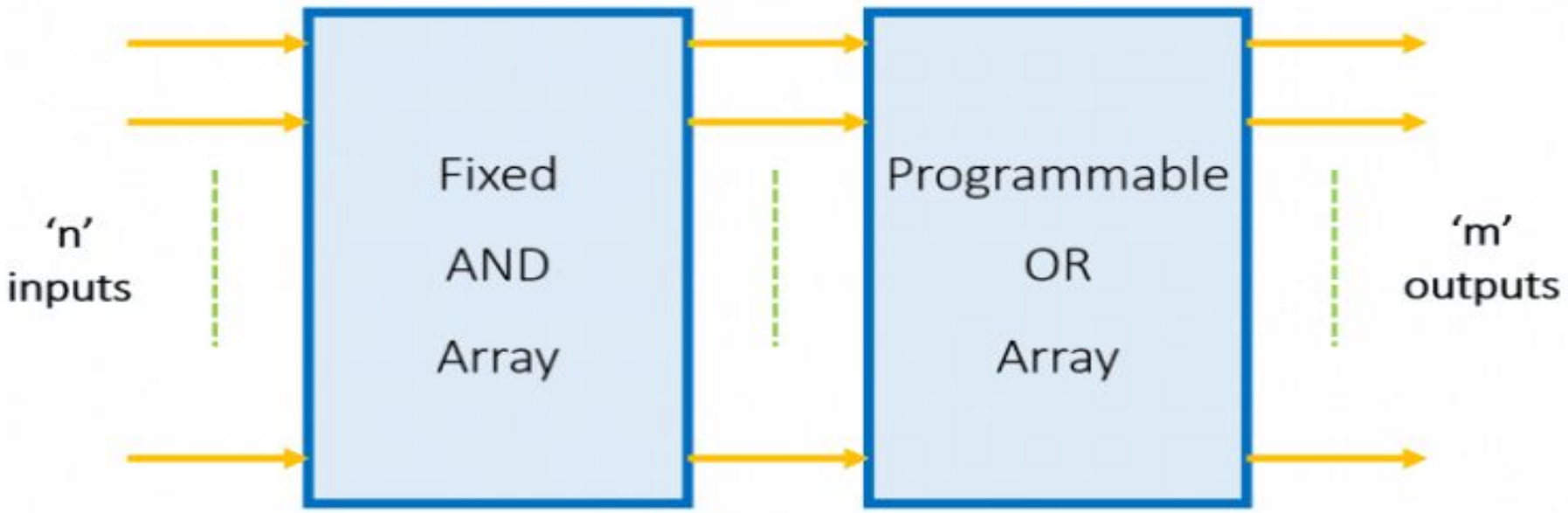
- A programmable ROM is a memory device which permanently stores binary information.
- It differs from a normal ROM because it can be programmed electrically once using a PROM programmer.
- This programmable logic device has Fixed AND arrays and Programmable OR arrays.



Block Diagram of PROM

- The inputs of the AND array are not programmable. Thus, we generate 2n product terms using 2n AND gates having n inputs each, using n x 2n decoder.
- This decoder generates **'n' minterms**.
- Unlike the AND array, the OR gates inputs are programmable.
- The output of the AND array are the inputs to this OR array system.
- Thus, the output of PROM will be as a **sum of minterms.**

PROM as PLD

PROM has a set a fixed AND gates and programmable OR array.

Here the AND array is used as a decoder and the programmable OR array implements the Boolean functions in sum of minterms.

To illustrate the basic idea of how the PROM can used as PLD, let us consider the following 16x4 PROM. The inputs A,B,C and D are decoded by the AND array to produce 16 products(locations).

The connections from inputs to the AND array are fixed as indicated by dots(.). On the other hand, the connections from the AND array outputs to the OR array are programmable (fusible links) as indicated by cross marks(x).

A designer can implement only four functions Y0,Y1,Y2 and Y3 with this PLD.

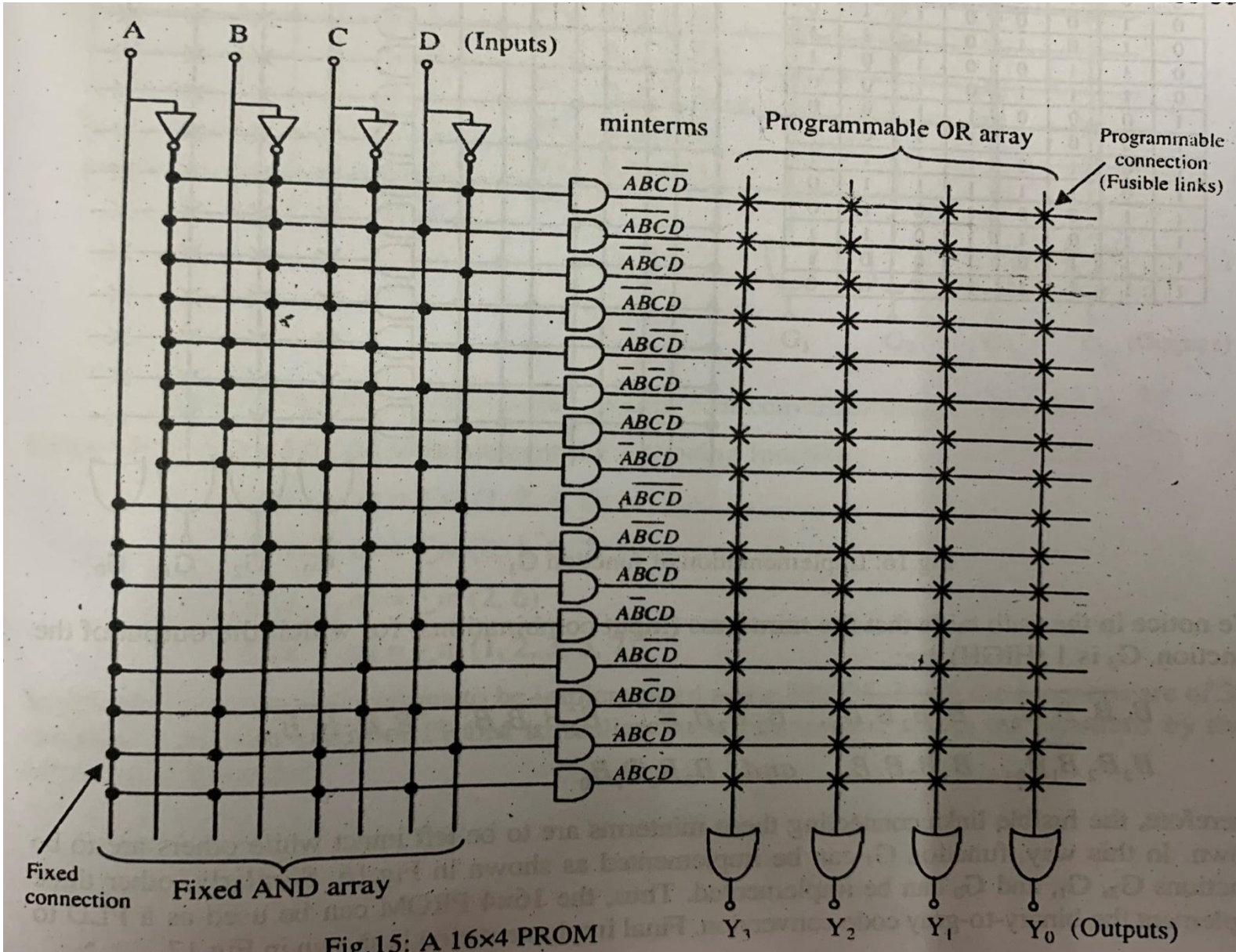


Fig.15: A 16×4 PROM

To implement a logic function , express it by a truth table to find out the minterms (input combinations) for which the output of the function is 1. Then OR these minterms, that is the fusible links connecting these minterms are left intact while others are to be blown.

Example 1: Show how the PROM can be used to perform the binary to gray code conversion.

<u>Solution:</u>

Truth table of a 4-bit Binary to Gray code conversion:

Truth table of a 4-bit Binary-to-Gray conversion

Binary Input				Gray code output			
B_3	B_2	B_1	B_0	G_3	G_2	G_1	G_0
0	0	0	0	0	0	0	0
0	0	0	1	0	0	0	1
0	0	1	0	0	0	1	1
0	0	1	1	0	0	1	0
0	1	0	0	0	1	1	0
0	1	0	1	0	1	1	1
0	1	1	0	0	1	0	1
0	1	1	1	0	1	0	0
1	0	0	0	1	1	0	0
1	0	0	1	1	1	0	1
1	0	1	0	1	1	1	1
1	0	1	1	1	1	1	0
1	1	0	0	1	0	1	0
1	1	0	1	1	0	1	1
1	1	1	0	1	0	0	1
1	1	1	1	1	0	0	0

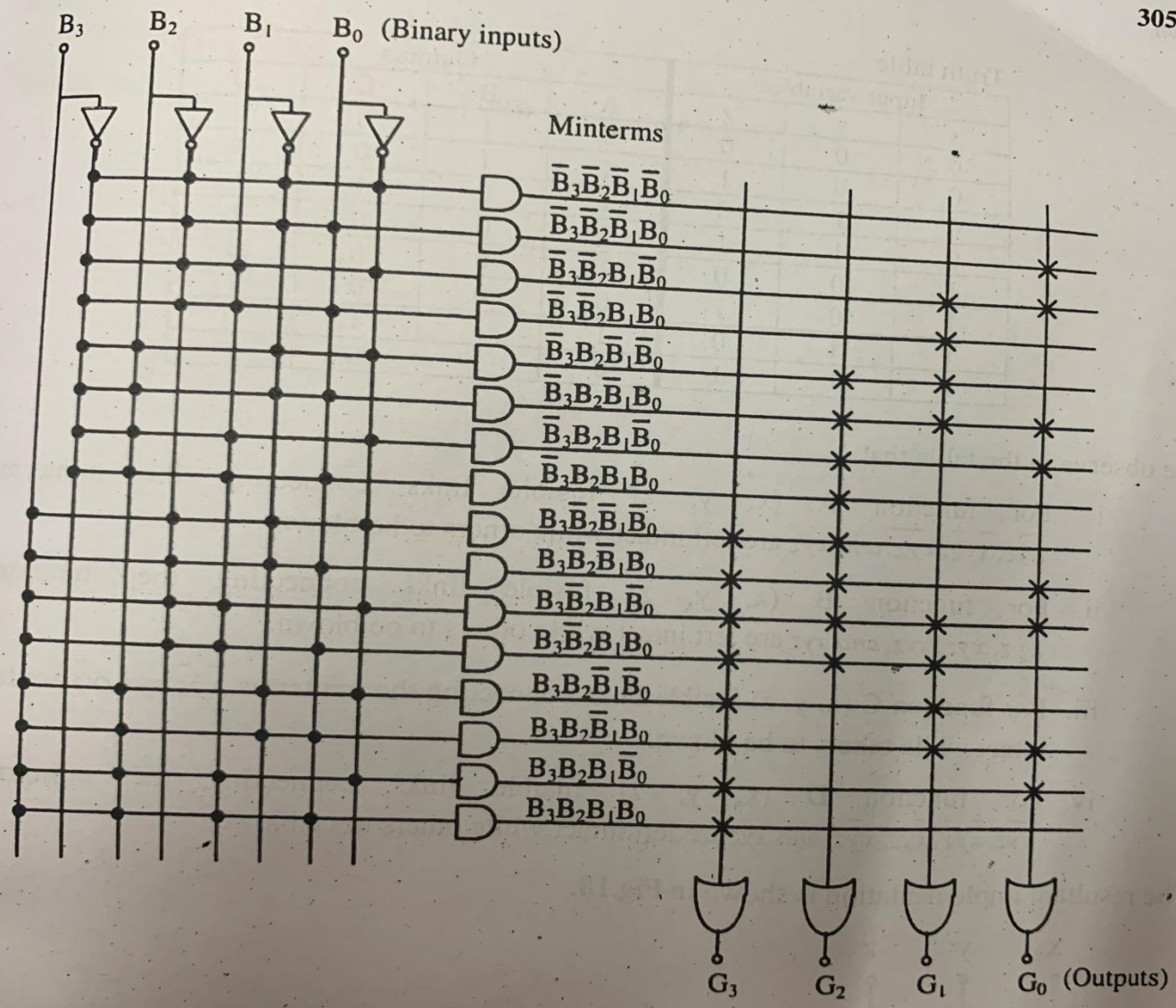


Fig.17: Implementation of Binary-to-Gray code conversion using a 16×4 PROM

Example 2. Use a PROM to implement the following functi

Example: $A(x,y,z) = \sum m\,(1,2,4,6)$

$B(x,y,z) = \sum m(0,1,6,7)$

$C(x,y,z) = \sum m(2,6)$

$D = \sum m(1,2,3,5,7)$

Truth table

Input variables			Outputs			
x	y	z	A	B	C	D
0	0	0	0	1	0	0
0	0	1	1	1	0	1
0	1	0	1	0	1	1
0	1	1	0	0	0	1
1	0	0	1	0	0	0
1	0	1	0	0	0	1
1	1	0	1	1	1	0
1	1	1	0	1	0	1

We observe in the table that,

i. For function A (x, y, z) fusible links connecting the minterms $\bar{x}\bar{y}\bar{z}, \bar{x}\bar{y}\bar{z}, \bar{x}\bar{y}\bar{z}$, *and* $xy\bar{z}$ are left intact while others to be blown.

ii. For function B (x, y, z) fusible links connecting the minterms $\bar{x}\bar{y}\bar{z}, \bar{x}\bar{y}\bar{z}, x\bar{y}\bar{z}$, *and* xyz are left intact while others to be blown.

iii. For function C (x, y, z) fusible links connecting the minterms $\bar{x}\bar{y}\bar{z}$ *and* $xy\bar{z}$ are left intact while others to be blown.

iv. For function D (x, y, z) fusible links connecting the minterms $\bar{x}\bar{y}\bar{z}, \bar{x}\bar{y}\bar{z}, \bar{x}\bar{y}\bar{z}, x\bar{y}\bar{z}$, *and* xyz are left intact while others to be blown.

The resultant implementation is shown in Fig.18.

x y z

The resultant implementation is shown in Fig.18.

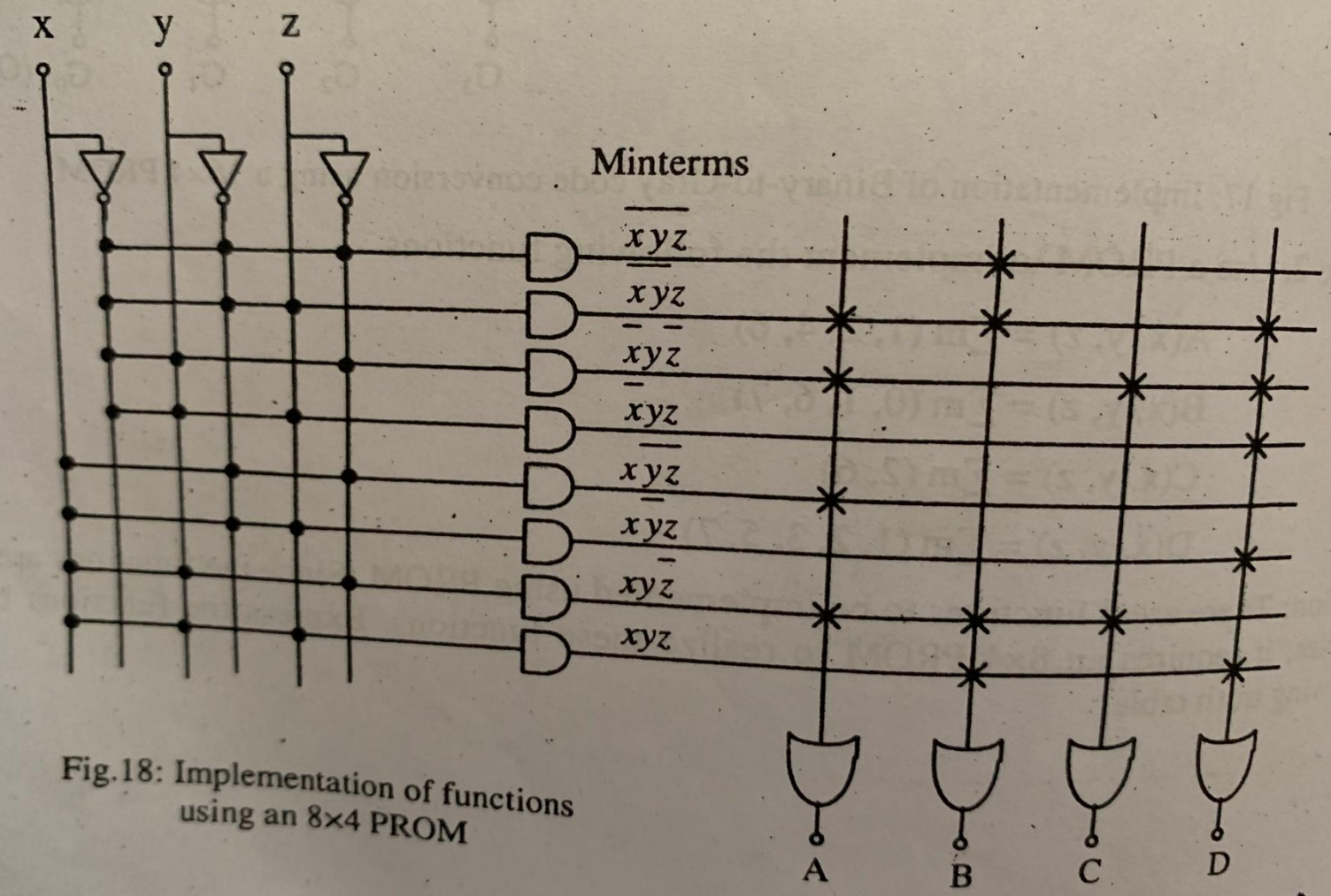
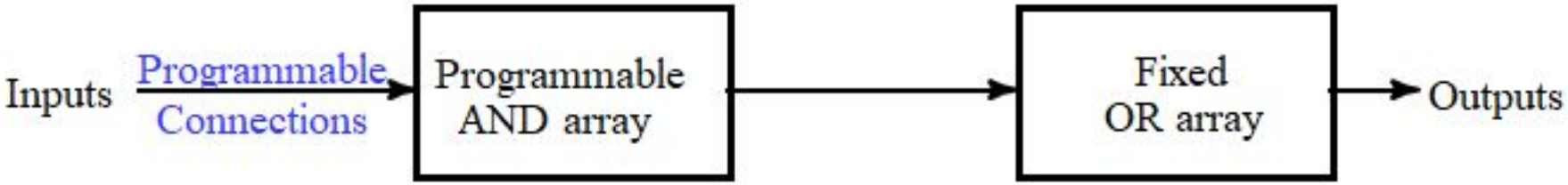


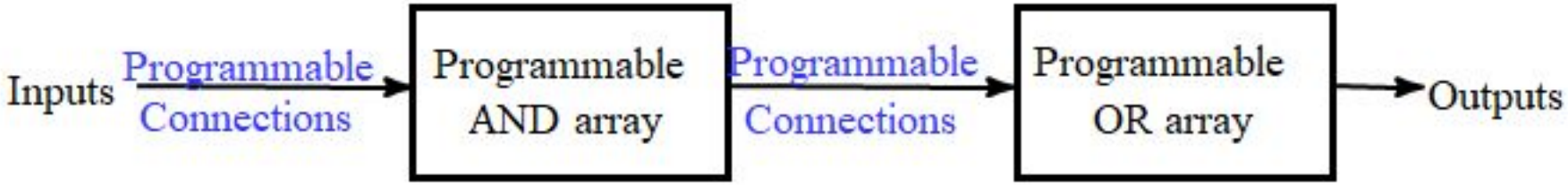
Fig.18: Implementation of functions using an 8×4 PROM

ROM, PAL and PLA Configurations

(a) Programmable read-only memory (PROM)



(b) Programmable array logic (PAL) device



(c) Programmable logic array (PLA) device